

Resumo de Segurança Informática

Índice Temático

1. Fundamentos de Segurança Informática
 2. Introdução à Criptografia e Cifras Históricas
 3. Hierarquia de Mecanismos Criptográficos
 4. Criptografia Simétrica e Cifras de Bloco
 5. Modos de Operação
 6. Esquemas MAC
 7. Funções Hash Criptográficas
 8. Criptografia Assimétrica e RSA
 9. Assinatura Digital
 10. Java Cryptography Architecture (JCA)
 11. Infraestruturas de Chave Pública (PKI)
 12. JCA e Certificados em Java
 13. Protocolo TLS
 14. Autenticação Baseada em Passwords
 15. Protocolo HTTP e Cookies
 16. Autenticadores em Contexto Web
 17. OAuth 2.0
 18. OpenID Connect
 19. Ataques ao OAuth 2.0 e Contramedidas
 20. Controlo de Acesso e Modelos
 21. Biblioteca Casbin
 22. Termos-Chave
-

1. Fundamentos de Segurança Informática

1.1 Definição de Segurança

No contexto da Informática, **segurança** refere-se à **proteção de informação**. O termo "informação" assume um sentido amplo, podendo denotar dados quaisquer, dispositivos que armazenam ou processam tais dados, ou recursos necessários à sua obtenção, armazenamento ou processamento.

A segurança informática lida com a **prevenção de ações não autorizadas** e, quando a prevenção falha, com a **deteção** dessas ações.

1.2 As Três (ou Quatro) Propriedades Básicas — CIA

Confidencialidade

Impedir a divulgação não autorizada da informação. Apenas entidades autorizadas devem ter acesso ao conteúdo.

- Distinção importante: **confidencialidade** foca em impedir a divulgação não autorizada; **privacidade** diz respeito à definição de quem tem acesso a quais informações.

Integridade

Uma informação íntegra é aquela que **não sofreu nenhuma alteração não autorizada**. O termo "alteração" é amplo: inclui modificações, supressões, inserções e reordenações.

- **Autenticidade** (ou integridade de origem): para além de não ter sido alterada, a informação deve ter sido gerada pela entidade que se afirma autora.
- **Não-repúdio**: consequência da autenticidade; a entidade que criou a informação não pode negar tê-lo feito.
- A autenticidade é frequentemente considerada um **quarto princípio** separado.

Disponibilidade

Propriedade de que um sistema, recurso ou dado **estará acessível sempre que necessário** pelo utilizador. É, possivelmente, a propriedade mais difícil de assegurar.

1.3 Tipos de Ataques

Um **ataque** é uma tentativa deliberada de **quebrar uma das propriedades** desejáveis de segurança.

Classificação: Passivos vs. Ativos

- **Ataques passivos**: o atacante apenas observa a informação sem a alterar.
- **Ataques ativos**: o atacante interage efetivamente com o sistema (modifica, injeta mensagens).

Ataques Específicos

Ataque	Tipo	Descrição
Divulgação de conteúdo (<i>information disclosure</i>)	Passivo	O atacante acede a informação à qual não devia ter acesso (ex: <i>sniffing</i>)
Análise de tráfego	Passivo	Análise de características do tráfego sem capturar o conteúdo (ex: tamanho/frequência de pacotes)
Disfarce (<i>spoofing</i>)	Ativo	O atacante passa-se por uma entidade legítima
Repasse / Repetição (<i>replay</i>)	Ativo	Captura e reenvio de mensagens legítimas em momento oportuno
Man-In-The-Middle (MITM)	Ativo	O atacante interpõe-se entre Alice e Bob, podendo ler e alterar mensagens em ambos os sentidos
Negação de serviço (DoS)	Ativo	Interrupção do serviço, frequentemente por <i>flooding</i> ; variante distribuída: DDoS

Convenção de nomes: *Alice* e *Bob* = entidades legítimas; *Eva* e *Trudy* = atacantes.

1.4 Proteção da Informação — Níveis

A segurança pode ser discutida em vários níveis:

- **Hardware:** módulos criptográficos, processadores com instruções especializadas, módulos de memória.
- **Software:** isolamento de processos pelo SO, bibliotecas de criptografia, *firewalls*, sistemas de detecção de intrusões.
- **Dados:** criptografia de ficheiros e bases de dados, passwords.
- **Comunicação:** criptografia de ligações individuais ou fim-a-fim.

2. Introdução à Criptografia e Cifras Históricas

2.1 Conceitos Básicos

Criptografia: ciência relativa à escrita de mensagens cifradas.

Componentes de um método criptográfico:

- **Texto plano** (m): mensagem original.
- **Criptograma / Texto cifrado** (c): versão protegida.
- **Função de cifra** $E(.)$: transforma m em c .
- **Função de decifra** $D(.)$: transforma c em m' .
- **Chave criptográfica** (k): o segredo da comunicação.
- **Funções geradoras de chave** $G(.)$: derivam chaves a partir de um segredo base.

Nota terminológica: Criptografia $\neq$ Criptoanálise $\neq$ Criptologia. Criptoanálise = métodos para quebrar códigos sem conhecer a chave. Criptologia = universo de ambos.

2.2 Cifras Históricas

Cifra de César

- Cifra de substituição com **deslocamento fixo** das letras do alfabeto.
- **Funções:**
 - $E(k)(m_i) = (m_i + k) \bmod 26 = c_i$
 - $D(k)(c_i) = (c_i - k) \bmod 26 = m_i$
- **Vulnerabilidade:** apenas 26 chaves possíveis → **força bruta trivial**.

Cifra de Substituição Monoalfabética

- Extensão da Cifra de César: cada letra tem um deslocamento específico (chave = permutação do alfabeto).
- **Espaço de chaves:** $26! \approx 4 \times 10^{26}$ → força bruta inviável manualmente.
- **Vulnerabilidade: análise de frequência** (ex: em Português, "A" aparece em 14% das palavras; "Y" em 0,01%).
- Um mesmo símbolo no texto plano mapeia sempre para o mesmo símbolo no criptograma → padrões exploráveis.

Cifra de Vigenère (Polialfabética)

- Utiliza uma **chave** (sequência de letras) e uma **tabula recta**.

- A mesma letra no texto plano pode mapear para letras diferentes, dependendo da posição.
- **Funções:**
 - $E(k)(m_i) = (m_i + k_{\{i \bmod x\}}) \bmod 26 = c_i$
 - $D(k)(c_i) = (c_i - k_{\{i \bmod x\}}) \bmod 26 = m_i$
 - onde x = tamanho da chave.
- Impede (ou dificulta) análises de frequência simples.

Máquina Enigma

- Dispositivo eletromecânico com rotores que definiam mapeamentos polialfabéticos.
- Chave = posição inicial dos rotores (alterada diariamente).
- Combinações possíveis: $\sim 10^{20} \rightarrow$ força bruta inviável.
- **Criptanálise bem-sucedida:** explorou limitações (uma letra nunca mapeava para si mesma, estrutura previsível das mensagens como "Heil Hitler"). Alan Turing e Gordon Welchman criaram a máquina **Bombe** para automatizar o processo.

2.3 Ataques de Força Bruta

- Testar todas as chaves possíveis é sempre teoricamente possível.
- Em métodos modernos, chaves de centenas ou milhares de bits tornam o número de combinações **proibitivo**.

3. Hierarquia de Mecanismos Criptográficos

3.1 Três Níveis de Abstração

```
Protocolos Criptográficos
  ↓ (usam)
Esquemas Criptográficos
  ↓ (usam)
Primitivas Criptográficas
```

Primitivas Criptográficas

- Algoritmos ou operações matemáticas básicas de cifra/decifra.
- Têm limitações fortes que impedem uso geral sem métodos auxiliares.
- Exemplos: **DES**, **AES** (cifras de bloco), **RSA**.

Esquemas Criptográficos

- Primitiva + métodos auxiliares para uma **tarefa criptográfica** específica.
- Associados a uma tarefa: **cifra** (confidencialidade) ou **assinatura digital / MAC** (integridade/autenticidade).
- Exemplo de nomenclatura: **DES-CBC-PKCS5Padding**.
- **A mesma primitiva pode ser usada em diferentes esquemas para diferentes tarefas.**

Protocolos Criptográficos

- Um ou mais esquemas + sequências de ações a realizar pelas entidades envolvidas.
- Exemplo: **TLS** usa RSA (assimétrico) na fase inicial e AES (simétrico) para os dados.

3.2 Classificação: Simétrico vs. Assimétrico

	Simétrico	Assimétrico (Chave Pública)
Chaves	Uma chave partilhada (k)	Par de chaves: pública (k_{pub}) + privada (k_{priv})
Cifra	Chave secreta para cifrar e decifrar	Cifra com chave pública; decifra com chave privada
Integridade	MAC	Assinatura Digital
Desempenho	Mais leve (preferível para grandes volumes)	Mais pesado (2+ ordens de grandeza mais lento)
Estabelecimento de chave	Desafiante (canal seguro ou esquema assimétrico)	Mais simples (chave pública pode ser divulgada)

4. Criptografia Simétrica e Cifras de Bloco

4.1 Definição Formal de Cifra Simétrica

Um esquema de cifra simétrica é definido por três componentes:

- **G(.)**: função de geração de chave (aleatória).
- **E(.)**: função de cifra: ($chave, mensagem$) $\rightarrow$ $criptograma$, suporta $m \in \{0,1\}^{**}$.
- **D(.)**: função de decifra: ($chave, criptograma$) $\rightarrow$ $mensagem$, suporta $c \in \{0,1\}^{**}$.

Processo de uso:

1. Gera-se chave k aleatória via $G(.)$.
2. Calcula-se $c = E(k)(m)$ e envia-se pelo canal inseguro.
3. O receptor calcula $m' = D(k)(c)$. Se não houve alterações, $m' = m$.

4.2 Propriedades

Correção

$$\forall m \in \{0,1\}^*, \forall k \in \text{Keys}: D(k)(E(k)(m)) = m$$

Segurança

- Deve ser **computacionalmente infazível** obter m a partir de c sem conhecer k .
- Força bruta é sempre teoricamente possível mas, com chaves de muitos bits, inviável na prática.

Limitações

- **Não garante integridade:** um atacante pode alterar bits do criptograma; o receptor decifra uma mensagem diferente sem o saber.
- **Não garante autenticidade:** o receptor não tem como saber se o criptograma é legítimo.

4.3 Cifras de Bloco

Operam sobre blocos de bits de **tamanho fixo**:

- **DES:** blocos de 64 bits, chaves de 56 bits (hoje **obsoleto**).
- **AES:** blocos de 128 bits, chaves de 128, 192 ou 256 bits (**standard atual do NIST**).

Internamente (exemplo DES): divisão, expansão, substituição por tabelas fixas — organizado em **rodadas** (*rounds*); o DES tem 16 rodadas.

Relação Tamanho de Bloco/Chave e Segurança

- **Chaves maiores** → mais difícil ataque por força bruta (cada bit adicional duplica as combinações).
- **Blocos maiores** → melhor ocultação de padrões estatísticos do texto plano.

Cifras de Fluxo (*Stream Ciphers*)

- Alternativa às cifras de bloco: operam sobre mensagens de tamanho efetivamente arbitrário sem necessidade de mecanismos adicionais.
- Adequadas para determinadas aplicações (ex: cifra em *links* de comunicação).

4.4 Boas Práticas na Escolha de Primitivas

- Optar por **standards internacionais** (ex: NIST) ou publicações académicas reconhecidas.
- Usar **implementações consolidadas e de confiança** (não implementar a própria criptografia).
- Evitar **segurança por obscuridade** (primitivas proprietárias tendem a ter mais vulnerabilidades por menor escrutínio público).
- Projetar sistemas com **fácil substituição** de primitivas (podem tornar-se obsoletas).

5. Modos de Operação

5.1 Definição e Objetivos

Modos de operação: algoritmos que utilizam repetidamente as funções de uma primitiva para cifrar mensagens de **tamanho arbitrário**.

Propriedades desejáveis:

- Cifrar mensagens de tamanho arbitrário.
- **Esconder padrões** do texto plano no criptograma.
- **Eficiência** computacional e tamanho do criptograma próximo do texto plano.
- **Acesso aleatório** (depende da aplicação).
- **Tolerância a erros / correção de erros** (depende da aplicação).

5.2 Modo ECB (Electronic Codebook)

- Divide m em blocos $m_1, m_2, \dots, m_L$ e cifra cada um **independentemente**: $c_i = E(k)(m_i)$.

Vantagens:

- Suporte razoável a **acesso aleatório**.
- **Resiliência a erros**: erros num bloco afetam apenas esse bloco.
- **Paralelização**: blocos podem ser cifrados/decifrados simultaneamente.

Desvantagem crítica:

- **Blocos iguais** → **criptogramas iguais** → revela padrões do texto plano.
- Exemplo clássico: cifrar uma imagem com ECB mantém formas e contornos visíveis.

5.3 Modo CBC (Cipher Block Chaining)

- Cifra em cadeia: cada bloco depende do resultado do bloco anterior.
- **Fórmula de cifra**: $c_i = E(k)(m_i \oplus c_{i-1})$, onde $c_0 = IV$.
- **Fórmula de decifra**: $m'_i = D(k)(c_i) \oplus c_{i-1}$.

Vetor de Inicialização (IV):

- Valor aleatório com o mesmo número de bits de um bloco.
- Elimina a correlação entre mensagens iguais cifradas com a mesma chave.
- **Não precisa de ser secreto**, mas deve ser **único e imprevisível**.
- Deve ser transmitido junto ao criptograma (ex: $IV \parallel c$).

Vantagens:

- **Esconde padrões** estatísticos muito melhor que o ECB (resultado parece ruído aleatório).

Desvantagens:

- **Propagação de erros**: erro num bloco c_j afeta a decifra de c_j e c_{j+1} .
- **Sem paralelização** na cifra (a decifra pode ser paralelizada).
- Acesso aleatório parcialmente limitado (decifra fácil, mas cifra de um bloco intermédio requer recifrar todos os seguintes).

Vulnerabilidade WEP: usava IV de 24 bits (só ~16M de IVs possíveis). O Paradoxo do Aniversário implica >50% de probabilidade de repetição após 5000 pacotes. Esta vulnerabilidade foi explorada para recuperar a chave em segundos.

5.4 Padding

Necessário quando o tamanho da mensagem não é múltiplo do tamanho do bloco.

PKCS#5

- Calcula o número X de **bytes** que faltam para completar o bloco.
- **Adiciona**: X bytes com valor X .
- Se $X = 0$ (tamanho já é múltiplo): adiciona n bytes com valor n (onde n = tamanho do bloco em bytes).
- **Remoção**: lê o valor Y do último byte e remove os últimos Y bytes.
- Trabalha sobre **bytes** (não bits).

Exemplo: mensagem de 664 bits com DES (64 bits/bloco):

- Sobram 24 bits = 3 bytes; faltam 5 bytes.
- Adiciona 5 bytes com valor **0x05**.

Padding Oracle Attack (Vaudenay, 2002):

- Explora o feedback do servidor sobre se o padding é válido ou não.
- Permite recuperar texto plano sem conhecer a chave.
- Ilustra conceitos de **ataque de texto cifrado escolhido** e **side channel attack**.

5.5 Modos de Operação em Stream

- Usam **apenas a função E(.)** da primitiva (tanto na cifra como na decifra).
- Geram um **keystream** (fluxo de chave) que é combinado com o texto plano por XOR bit-a-bit.
- **Não necessitam de padding**.

Processo de cifra:

1. Estado inicial $I_0 = IV$.
2. $ks_i = E(k)(I_i)$ (gera bloco do keystream).
3. Cifra: $c_i = m_i \oplus ks_i$; decifra: $m_i = c_i \oplus ks_i$.
4. Gera próximo estado I_{i+1} e repete.

Variantes (diferem na geração de I_{i+1}):

- **CFB:** próximo estado = bits de texto cifrado do estado atual.
- **OFB:** próximo estado = bits do keystream do estado atual.
- **CTR:** $I_{i+1} = I_i + 1$ (contador incrementado).

Modo CTR (Counter)

- Estado inicial: $I_0 = IV$; atualização: $I_{i+1} = I_i + 1$.
- **Erros não se propagam:** erro num bit do criptograma afeta apenas o bit correspondente.
- **Acesso aleatório simples:** para decifrar a partir do byte 802 (blocos de 8 bytes): iniciar com $I = IV + \lceil 802/8 \rceil = IV + 100$.
- **Segurança:** o IV nunca deve repetir com a mesma chave.

6. Esquemas MAC

6.1 Autenticação de Mensagens

Objetivo: o receptor deve ser capaz de validar que uma mensagem é **autêntica** (íntegra e de fonte legítima).

6.2 Definição Formal de MAC

Um esquema MAC (*Message Authentication Code*) é composto por:

- **G(.):** geração de chave simétrica k (aleatória).
- **T(.):** geração de marca: $t = T(k)(m)$.

- **V(.)**: verificação: $V(k)(t, m) \rightarrow \{true, false\}$.

Nota: se confidencialidade não é requisito, m e t podem ser transmitidos em **texto plano**.

6.3 Propriedades

Correção

$\forall m \in \{0,1\}^*, \forall k \in \text{Keys}: V(k)(T(k)(m), m) = true$

Esta definição **não impede** que mensagens diferentes possam ter a mesma marca (**colisões**), embora a probabilidade deva ser muito baixa.

Segurança (duas propriedades simultâneas)

1. **Resistência a falsificação seletiva:** dada uma mensagem m específica, deve ser computacionalmente infazível encontrar t tal que $V(k)(t, m) = true$ sem conhecer k .
2. **Resistência a falsificação existencial:** deve ser computacionalmente infazível encontrar qualquer par (t, m) tal que $V(k)(t, m) = true$ sem conhecer k .

Outras propriedades

- Suporta mensagens de tamanho arbitrário.
- A marca deve ter **tamanho fixo** (obtido com funções hash).
- **Não garante não-repúdio:** a chave é partilhada, logo não é possível determinar qual das partes legítimas gerou o par (*mensagem, marca*).

6.4 Cifra Autenticada

Para garantir simultaneamente confidencialidade e integridade/autenticidade, combinam-se cifra e MAC.

Encrypt-then-MAC

Transmite: $E(k_1)(m) \parallel T(k_2)(E(k_1)(m))$

- Receptor verifica a autenticidade do criptograma **antes** de decifrar.
- Mais eficiente (rejeita criptogramas inválidos imediatamente).
- Dificulta ataques como o *padding oracle attack*.
- **Boa prática:** se o criptograma usa IV, o MAC deve ser calculado sobre o conjunto $IV + \text{criptograma}$.

MAC-then-Encrypt

Transmite: $E(k_1)(m \parallel T(k_2)(m))$

- O receptor decifra primeiro, depois verifica a marca.

Nota: devem ser usadas **chaves diferentes** para a cifra e para o MAC.

6.5 Modo GCM (Galois Counter Mode)

Combina cifra e MAC num único modo:

- Combina o **modo CTR** com a função hash **GHASH**.
- Suporta **AAD** (*Additional Authentication Data*): dados não cifrados mas autenticados.
- Abordagem do tipo *encrypt-then-MAC*.
- Amplamente utilizado (ex: WPA3).
- A matemática subjacente baseia-se em operações no grupo finito GF(128) (daí a referência a Évariste Galois).

7. Funções Hash Criptográficas

7.1 Definição

Uma função de hash H mapeia sequências de bits de tamanho arbitrário para sequências de bits de tamanho fixo:

$$H: \{0,1\}^* \mapsto \{0,1\}^n$$

onde n é a **dimensão do hash**.

Toda função de hash é não-invertível (e não-injetiva): o domínio é infinito, o contra-domínio é finito (2^n valores possíveis).

7.2 Propriedades de um Bom Hash Criptográfico

Propriedade	Descrição
Eficiência	Deve ser computacionalmente fácil calcular $H(x)$ dado x
Resistência a segunda pré-imagem	Dado x , deve ser infazível encontrar $x' \neq x$ tal que $H(x') = H(x)$
Resistência a colisão	Deve ser infazível encontrar quaisquer $x \neq x'$ tal que $H(x) = H(x')$
Resistência a pré-imagem	Dado y , deve ser infazível encontrar x tal que $H(x) = y$

Relação: resistência a colisão implica resistência a segunda pré-imagem.

Efeito avalanche: pequenas alterações numa mensagem devem gerar grandes alterações no hash.

7.3 Funções Hash Populares

- **SHA-256:** 256 bits de output; 2^{256} imagens possíveis. Amplamente utilizado.
- **SHA-1:** 160 bits (hoje considerado obsoleto para muitos usos).
- **MD5:** 128 bits (considerado obsoleto por vulnerabilidades conhecidas).

7.4 Aplicações

- **Verificação de integridade:** comparar o hash de um ficheiro descarregado com o hash do ficheiro legítimo.
- **Componente de esquemas MAC** (ex: HMAC).
- **Geração de chaves a partir de passwords:** uma password é passada por uma função hash com dimensão adequada para gerar a chave.
- **Prova de trabalho** em *blockchains*.

7.5 HMAC

O HMAC (*Hashed MAC*) usa uma função hash H e uma chave simétrica k :

$$HMAC(k, m) = H((k \oplus opad) \parallel H((k \oplus ipad) \parallel m))$$

onde:

- $ipad = 0x3636 \dots 36$
- $opad = 0x5c5c \dots 5c$

Características:

- Aplicável a qualquer função hash (ex: *HMAC-SHA256*).
- Funciona com chaves de qualquer tamanho.
- Output de **tamanho fixo** (independente do tamanho da mensagem ou da chave).

8. Criptografia Assimétrica e RSA

8.1 Conceitos Fundamentais

Na criptografia assimétrica:

- Há **duas chaves diferentes**: pública (k_{pub}) e privada (k_{priv}).
- O que uma chave cifra, a outra decifra: $D(k_{priv})(E(k_{pub})(m)) = m$ e $D(k_{pub})(E(k_{priv})(m)) = m$.
- A **chave pública pode ser conhecida por todos** (inclusive atacantes).
- A **chave privada deve ser mantida em segredo** pela entidade que a gera.

Ponto crítico: nem a mesma chave decifra o que ela própria cifrou:

- $D(k_{pub})(E(k_{pub})(m)) \neq m$
- $D(k_{priv})(E(k_{priv})(m)) \neq m$

8.2 Uso das Chaves por Tarefa

Tarefa	Operação "privada"	Chave usada na proteção	Chave usada na desproteção
Cifra assimétrica	Decifra (só o destinatário decifra)	Chave pública do destinatário	Chave privada do destinatário
Assinatura digital	Assinar (só o autor assina)	Chave privada do autor	Chave pública do autor

8.3 Usos Típicos da Criptografia Assimétrica

- **Estabelecimento seguro de chaves simétricas** (esquema híbrido).
- **Assinatura digital**.
- **Não usada** para grandes volumes de dados (2+ ordens de grandeza mais lenta que a simétrica).

8.4 Definição Formal de Cifra Assimétrica

Composta por (G, E, D):

- **G(.)**: função probabilística que gera pares (k_{pub}, k_{priv}) com relação correta entre si.
- **E(.)**: cifra com k_{pub} ; o domínio são números inteiros positivos menores que N .
- **D(.)**: decifra com k_{priv} .

Correção: $\forall m \in M, \forall (k_e, k_d) \in \text{KeyPairs}: D(k_d)(E(k_e)(m)) = m$

Segurança: computacionalmente infazível obter m a partir de c mesmo conhecendo k_{pub} .

Limitações práticas:

- Mensagens têm dimensão máxima limitada (mensagens muito grandes necessitariam de múltiplas chamadas, o que seria computacionalmente proibitivo).
- Alta complexidade computacional.
- Uso típico: **esquemas híbridos** (cifra assimétrica para estabelecer chave simétrica, depois usa criptografia simétrica).

8.5 A Primitiva RSA

Proposta por Rivest, Shamir e Adleman na década de 1970.

Chaves

- **Chave pública:** (E, N)
- **Chave privada:** (D, N)

Operações

- **Cifra:** $C = M^E \bmod N$
- **Decifra:** $M = C^D \bmod N$
- Restrição: $M < N$ (para evitar ambiguidades).

Geração de Chaves

1. Escolher aleatoriamente dois primos grandes distintos P e Q .
2. Calcular $N = P \cdot Q$.
3. Calcular $\lambda(N) = \text{lcm}(P-1, Q-1)$ (Função de Carmichael).
4. Derivar E e D usando teoremas matemáticos.

Segurança

- A segurança baseia-se na dificuldade de **fatorar números grandes**.
- Um atacante que conhece N mas não P e Q não consegue calcular facilmente $\lambda(N)$.
- O **Algoritmo de Shor** (para computadores quânticos) resolve este problema em tempo polinomial, mas ainda há obstáculos para sua aplicação a valores de N do RSA real.
- Valores típicos de N : entre 2^{1023} e 2^{4095} .

Exemplo numérico: N=33, E=7, D=3

- Cifra de $M=8$: $C = 8^7 \bmod 33 = 2$
- Decifra de $C=2$: $M = 2^3 \bmod 33 = 8 \checkmark$

8.6 Formatação / Padding nas Cifras Assimétricas

Necessária para mapear mensagens (strings de bits) para números inteiros do domínio da função de cifra.

- **PKCS#1 v1.5**: método mais antigo.
- **OAEP** (*Optimal Asymmetric Encryption Padding*): método preferível, introduz aleatoriedade no padding.

Funções do padding:

- Codificação não ambígua da mensagem.
- Introdução de aleatoriedade para resistir a ataques.
- Evitar casos especiais (ex: no RSA, $M = 0$ sempre resultaria em $C = 0$).

8.7 Esquema Híbrido

1. Alice gera par (K_e, K_d) e envia K_e (pública) a Bob.
2. Bob gera uma **chave simétrica de sessão** k e cifra-a: $c_k = E(K_e)(k)$.
3. Alice decifra com K_d e obtém k .
4. A partir daí, usam um **esquema simétrico** com k .

9. Assinatura Digital

9.1 Conceito

A assinatura digital é o esquema criptográfico **assimétrico** para garantir integridade/autenticidade, análogo ao MAC (que é simétrico).

Diferenças em relação ao MAC:

- A assinatura é gerada com a **chave privada** (apenas o autor pode assinar).
- A verificação é feita com a **chave pública** (qualquer entidade pode verificar).
- Garante **não-repúdio** (ao contrário do MAC).
- As chaves assimétricas duram mais (anos/décadas) → adequadas para documentos de longo prazo.

9.2 Processo

Assinatura

1. Calcula-se o hash $h = H(m)$ com uma função hash criptográfico.
2. Aplica-se a função de cifra da primitiva assimétrica com a chave privada: $s = E(k_{priv})(h)$.

Verificação

1. Aplica-se a função de decifra com a chave pública do autor: $h' = D(k_{pub})(s)$.
2. Calcula-se o hash da mensagem recebida: $h'' = H(m')$.
3. Se $h' = h''$, a assinatura é válida; caso contrário, inválida.

9.3 Definição Formal

Composta por (G, S, V) :

- **G(.)**: geração de par de chaves (igual à cifra assimétrica).
- **S(.)**: $s = S(k_{priv})(m)$, suporta $m \in \{0,1\}^{**}$.
- **V(.)**: $V(k_{pub})(s, m) \rightarrow \{true, false\}$.

Correção: $\forall m \in \{0,1\}^{**}, \forall (k_{priv}, k_{pub}) \in \text{KeyPairs}: V(k_{pub})(S(k_{priv})(m), m) = true$

9.4 Segurança

- **Resistência a falsificação seletiva**: infazível encontrar s para uma mensagem m específica sem k_{priv} .
- **Resistência a falsificação existencial**: infazível encontrar qualquer par (m, s) válido sem k_{priv} .
- Depende tanto da qualidade do hash quanto da segurança da primitiva assimétrica.

9.5 Componentes de um Esquema de Assinatura Digital

Um esquema define três componentes:

1. **Função de hash** (ex: MD5, SHA-1, SHA-256).
2. **Método de formatação** (ex: PKCS#1 v1.5, PSS, X9.31).
3. **Primitiva assimétrica** (ex: RSA, DSA).

Exemplo de notação: **SHA256withRSA**.

Assinar $\neq$ Decifrar; Verificar $\neq$ Cifrar (objetivos e processos são diferentes).

10. Java Cryptography Architecture (JCA)

10.1 Visão Geral

A JCA é uma API criptográfica padronizada para Java que oferece:

- **Independência de algoritmos**: o código permanece quase idêntico ao mudar de algoritmo (basta alterar uma string).
- **Independência de implementação**: pode-se trocar a implementação (o *provider*) sem alterar o código.
- **Interoperabilidade** entre diferentes implementações e plataformas.

10.2 Arquitetura: Três Componentes

Cryptographic Service Providers (CSPs)

- Pacotes/classes com implementações concretas dos algoritmos.
- Exemplos incluídos no JDK: **Sun**, **SunRsaSign**, **SunJCE**.
- Podem ser instalados *providers* de terceiros.
- Registados no ficheiro **java.security** com prioridades, ou dinamicamente via classe **Security**.
- **Recomendação**: não especificar um *provider* no código (aumenta portabilidade e facilita atualizações).

Engine Classes

- Classes que representam abstratamente esquemas criptográficos.
- Instanciadas via método estático `getInstance()`.
- São **opacas** (escondem detalhes internos).
- Exemplos principais:

Engine Class	Uso
<code>Cipher</code>	Cifra simétrica e assimétrica
<code>Mac</code>	Esquemas MAC
<code>Signature</code>	Assinatura digital
<code>MessageDigest</code>	Funções hash
<code>KeyGenerator</code>	Geração de chaves simétricas
<code>KeyPairGenerator</code>	Geração de pares de chaves assimétricas
<code>SecureRandom</code>	Geração de números pseudo-aleatórios

Specification Classes

- Representações **transparentes** de parâmetros (ex: chaves, IVs).
- Permitem leitura/escrita de valores específicos.
- Exemplos: `IvParameterSpec`, `RSAPublicKeySpec`, `SecretKeySpec`, `RSAPublicKeySpec`.

10.3 Strings de Transformação

Identificam o esquema criptográfico no `getInstance()`. Não são case-sensitive.

Engine Class	Formato	Exemplo
<code>Cipher</code>	"algoritmo/modo/padding" ou "algoritmo"	"AES/CBC/PKCS5Padding"
<code>Mac</code>	"Hmac" + hashName	"HmacSHA256"
<code>Signature</code>	"<hash>with<primitiva>"	"SHA256withRSA"
<code>KeyGenerator</code>	Nome da primitiva	"AES"
<code>KeyPairGenerator</code>	Nome da primitiva	"RSA"

10.4 Engine Class `Cipher`

Inicialização via `init(mode, key)`:

- `Cipher.ENCRYPT_MODE` — cifra.
- `Cipher.DECRYPT_MODE` — decifra.
- `Cipher.WRAP_MODE` / `Cipher.UNWRAP_MODE` — cifra/decifra de chaves.

Métodos principais:

- `update(byte[]) → byte[]`: cifra/decifra incremental (quando os dados chegam em partes).
- `doFinal(byte[]) → byte[]`: **sempre deve ser chamado** para finalizar o processo.

- `wrap(Key) → byte[]`: cifra uma chave.
- `unwrap(byte[], ...) → Key`: decifra uma chave.

Exemplo básico:

```
Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
cipher.init(Cipher.ENCRYPT_MODE, key);
byte[] bytes = cipher.doFinal(msg.getBytes());
```

10.5 Manipulação de Chaves

- `SecretKey` / `PublicKey` / `PrivateKey`: interfaces para tipos de chaves (opacas).
- `KeyPair`: par de chaves pública e privada.
- `KeyGenerator.generateKey()` → gera chave simétrica.
- `KeyPairGenerator.generateKeyPair()` → gera par de chaves assimétricas.
- `KeyFactory` / `SecretKeyFactory`: convertem entre representações opacas e transparentes.
- `SecretKeySpec`: cria chave simétrica a partir de bytes específicos.

10.6 Classes Stream da JCA

Para integração com *streams* de I/O:

- `CipherInputStream` / `CipherOutputStream`: aplicam cifra/decifra a *streams*.
- `DigestInputStream` / `DigestOutputStream`: calculam hash a *streams*.

Uso: instanciar com o *stream* subjacente e um objeto `Cipher`/`MessageDigest` já configurado. Chamar `close()` no final (para `Cipher`) ou `digest()` (para `MessageDigest`).

10.7 Engine Class `Mac`

```
Mac mac = Mac.getInstance("HmacSHA256");
mac.init(key); // não tem parâmetro de modo
mac.update(data); // não retorna nada
byte[] tag = mac.doFinal(someImportantMessage.getBytes());
```

10.8 Engine Class `Signature`

```
// Assinatura
Signature sign = Signature.getInstance("SHA256withRSA");
sign.initSign(privKey);
sign.update(m.getBytes());
byte[] s = sign.sign();

// Verificação
sign.initVerify(pubKey);
```



```
sign.update(m.getBytes());  
boolean valid = sign.verify(s);
```

11. Infraestruturas de Chave Pública (PKI)

11.1 O Problema da Autenticação de Chaves Públicas

Embora a chave pública não precise de confidencialidade, o seu envio pelo canal inseguro é vulnerável a um ataque **Man-In-The-Middle**:

1. Eva intercepta a chave pública de Alice e substitui-a pela sua própria.
2. Bob cifra mensagens com a chave de Eva, julgando ser a de Alice.
3. Eva decifra, lê (e eventualmente re-cifra com a chave real de Alice).

Solução: garantir a autenticidade das chaves públicas através de **certificados digitais**.

11.2 Certificados Digitais e Autoridades de Certificação

Um **certificado digital** associa de forma verificável uma **chave pública** à **identidade** da entidade proprietária.

Uma **Autoridade de Certificação (AC/CA)** é uma entidade de confiança que:

1. Verifica a identidade da entidade solicitante (trâmites burocráticos).
2. Emite o certificado associando identidade e chave pública.
3. **Assina digitalmente** o certificado com a sua própria chave privada.

Analogia: os CAs são como **cartórios notariais** do mundo digital.

11.3 Cadeia de Certificação

Problema: para verificar o certificado de Alice, Bob precisa da chave pública da CA. Mas como obtê-la de forma segura?

Solução: cadeias de certificação. Cada CA tem o seu certificado emitido por outra CA, criando uma cadeia. No final da cadeia está o **certificado raiz** (auto-assinado).

Certificado raiz (auto-assinado):

- Os campos **subject** e **issuer** são iguais.
- Considerados inseguros por si mesmos, mas **implicitamente confiáveis** (pré-instalados no sistema — *trust anchors*).
- Vêm incluídos em sistemas operativos, browsers e outras aplicações.

Processo de verificação (recursivo):

1. Verificar a assinatura do certificado folha usando a chave pública da CA emissora.
2. Verificar o certificado dessa CA usando a chave da CA acima.
3. Continuar até chegar a um certificado raiz de confiança (*trust anchor*).

Falha na verificação se:

- Alguma assinatura ao longo do caminho é incorreta.
- O caminho está incompleto.
- O certificado raiz não é de um *trust anchor* reconhecido.

11.4 Certificados X.509 e ASN.1

O **X.509** (standard da ITU) define a estrutura dos certificados digitais usando a linguagem **ASN.1** (*Abstract Syntax Notation One*).

Estrutura de um certificado X.509 (simplificada)

```
Certificate ::= SEQUENCE {  
    tbsCertificate TBSCertificate,  -- corpo a ser assinado  
    signatureAlgorithm AlgorithmIdentifier,  
    signatureValue BIT STRING  
}
```

Campos do TBSCertificate (principais)

- **version**: versão do X.509 (v1, v2, v3).
- **serialNumber**: número único dentro de uma CA.
- **issuer**: identidade da CA emissora.
- **validity**: datas de início e fim de validade.
- **subject**: identidade do detentor da chave pública.
- **subjectPublicKeyInfo**: chave pública e algoritmo.
- **extensions** (apenas v3): informações adicionais.

Extensões PKIX Importantes

- **Key Usage**: usos permitidos para as chaves (ex: assinatura de certificados, apenas para CAs).
- **Subject Alternative Names**: identificadores alternativos (ex: endereços de email, IPs, hostnames DNS).
- **Basic Constraints**: indica se é certificado de CA; comprimento máximo do caminho.
- **CRL Distribution Points**: onde obter a **CRL** (*Certificate Revocation List*).

OIDs (*Object Identifiers*)

Esquema hierárquico de nomeação (ISO/ITU) para identificar algoritmos e entidades. Exemplo: RSA = **1.2.840.113549.1.1.1**.

Codificação

- **DER** (*Distinguished Encoding Rules*): regras de codificação binária para ASN.1.
- **PEM**: certificados codificados em Base64, armazenados em ficheiros **.pem**, **.cer**, **.crt**.

- **PFX/PKCS#12**: container cifrado (protegido por password) para chave privada + certificado(s); ficheiro `.pfx`. Formato binário.

Importante: certificados **nunca armazenam chaves privadas**; apenas as públicas.

11.5 Revogação de Certificados

Um certificado pode ser revogado antes da expiração (ex: comprometimento da chave privada).

CRL (*Certificate Revocation List*): lista mantida pelas CAs com os certificados revogados. Ao receber um certificado, deve-se verificar se não está na CRL.

12. JCA e Certificados em Java

12.1 Classe `Certificate` e `CertificateFactory`

- **`Certificate`**: representação abstrata de um certificado.
- **`X509Certificate`**: extensão concreta para certificados X.509; implementa `X509Extension`.

Instanciação via `CertificateFactory`:

```
CertificateFactory cf = CertificateFactory.getInstance("X.509");
FileInputStream in = new FileInputStream("certificado.cer");
X509Certificate cert = (X509Certificate) cf.generateCertificate(in);
```

Suporta certificados X.509 com codificação DER (binário ou Base64/PEM).

Métodos úteis de `X509Certificate`:

- `getSubjectX500Principal()` / `getIssuerX500Principal()`: nome do sujeito/emissor.
- `getPublicKey()`: chave pública (objeto `PublicKey`).
- `checkValidity()`: verifica se está dentro do período de validade.
- `getVersion()`, `getSigAlgName()`, `getNotBefore()`, `getNotAfter()`, `getSignature()`.

12.2 Key Stores

`KeyStore`: repositório de material criptográfico (chaves e certificados).

Tipos suportados: JKS, JCEKS, PKCS12.

Tipos de entradas:

- **`PrivateKeyEntry`**: chave privada + certificado(s).
- **`SecretKeyEntry`**: chave simétrica.
- **`TrustedCertificateEntry`**: certificado raiz de confiança.

```
KeyStore ks = KeyStore.getInstance("PKCS12");
ks.load(new FileInputStream("certs.pfx"), "changeit".toCharArray());
```

Métodos principais: `aliases()`, `getCertificate(alias)`, `getKey(alias, password)`, `store()`, `getEntry()`, `isCertificateEntry()`, `isKeyEntry()`, `setEntry()`.

Proteção por password: a KeyStore tem uma password para integridade do repositório inteiro; cada entrada com material confidencial tem a sua própria password.

12.3 Validação de Cadeias: `CertPathValidator`

Método 1: Construção manual do caminho

```
List<X509Certificate> certList = new ArrayList<>();
certList.add(0, certFolha);
certList.add(1, certIntermedio);
CertPath cp = cf.generateCertPath(certList);
```

Configurar Trust Anchors e Parâmetros

```
Set<TrustAnchor> anch = new HashSet<>();
anch.add(new TrustAnchor(certRaiz, null));
PKIXParameters params = new PKIXParameters(anch);
params.setRevocationEnabled(false);
```

Validar

```
CertPathValidator cpv = CertPathValidator.getInstance("PKIX");
PKIXCertPathValidatorResult result = (PKIXCertPathValidatorResult)
cpv.validate(cp, params);
```

- Sucesso: retorna `PKIXCertPathValidatorResult` com `getPublicKey()`.
- Falha: lança `CertPathValidatorException` com `getIndex()` (índice do certificado problemático).

12.4 Validação de Cadeias: `CertPathBuilder`

Quando não se conhece a ordem/composição exata do caminho, usa-se `CertPathBuilder`.

```
// 1. Criar CertStore com todos os certificados
CertStore certStore = CertStore.getInstance("Collection",
    new CollectionCertStoreParameters(certCollection));

// 2. Definir qual certificado folha queremos validar
X509CertSelector certSelector = new X509CertSelector();
certSelector.setCertificate(certFolha);

// 3. Configurar parâmetros
```

```
PKIXBuilderParameters params = new PKIXBuilderParameters(trustAnchorSet,
certSelector);
params.addCertStore(certStore);
params.setRevocationEnabled(false);

// 4. Construir o caminho
CertPathBuilder builder = CertPathBuilder.getInstance("PKIX");
CertPathBuilderResult result = builder.build(params);
CertPath certPath = result.getCertPath();
```

13. Protocolo TLS

13.1 Histórico e Versões

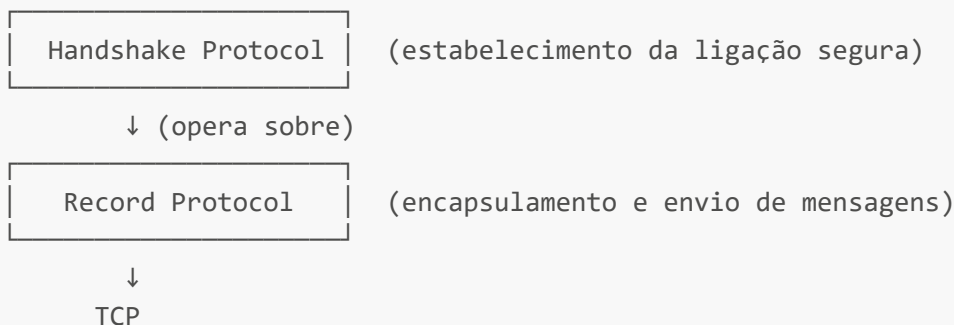
- **SSL** (Secure Sockets Layer): desenvolvido pela Netscape no início dos anos 90. Versões 1.0 e 2.0 com falhas graves; 3.0 redesenhado.
- **TLS 1.0** (RFC 2246, 1999): baseado no SSL 3.0, sem retrocompatibilidade.
- **TLS 1.1, 1.2**: evoluções progressivas.
- **TLS 1.3** (RFC 8446, 2018): eliminou esquemas menos seguros (ex: AES-CBC não autenticado, métodos sem *Perfect Forward Secrecy*).

13.2 Objetivos

- **Canal de comunicação seguro** sobre rede insegura entre dois *endpoints*.
- Garante: **confidencialidade**, **integridade** e **autenticação** (de pelo menos um *endpoint*).
- Posicionado entre camadas de Aplicação e Transporte na pilha TCP/IP.
- Opera sobre TCP (versão padrão) ou UDP (DTLS — *Datagram TLS*).

13.3 Sub-protocolos

O TLS divide-se em dois sub-protocolos:



13.4 Record Protocol

Responsabilidades: fragmentar dados em registos, aplicar esquemas criptográficos, enviar.

Campos do header de um registo TLS:

- **Content-Type:** tipo de mensagem (aplicação, alerta, handshake, ...).
- **Length:** tamanho da mensagem (incluindo MAC tags e padding).
- **Protocol Message:** a mensagem em si (possivelmente cifrada).

Número de sequência: não incluído no header, mas usado no cálculo da tag de autenticação (para prevenir ataques de repetição e reflexão).

Material criptográfico por direção:

- Chaves, IVs e números de sequência diferentes para cada sentido:
 - `client_write_key`, `client_write_iv` (cliente → servidor).
 - `server_write_key`, `server_write_iv` (servidor → cliente).

Proteção no TLS 1.2: suportava cifras não autenticadas com MAC-then-encrypt separado.

Proteção no TLS 1.3: apenas **cifras autenticadas** (ex: AES-GCM); o número de sequência é incluído como AAD.

Compressão: removida no TLS 1.3 (vulnerável ao ataque **CRIME**).

Proteção Contra Ataques de Repetição

- Cada registo tem número de sequência único → a tag inclui este número.
- Um registo reenviado terá número de sequência errado → falha na verificação.

Proteção Contra Ataques de Reflexão

- Material criptográfico diferente para cada direção → mensagens trocadas de direção falham na verificação.

13.5 Cipher Suite

Define o conjunto de esquemas criptográficos para a comunicação. O TLS suporta **vários** (negociados no handshake).

Vantagens de múltiplas cipher suites:

- Imunidade a futuras vulnerabilidades (evitar suites afetadas).
- Retrocompatibilidade entre versões.

Formato (exemplo TLS 1.2): `TLS_RSA_WITH_3DES_EDE_CBC_SHA`

- RSA: estabelecimento de chave.
- 3DES_EDE + CBC: cifra simétrica.
- SHA: hash para HMAC.

Formato TLS 1.3: ex. `TLS_AES_128_GCM_SHA256` (apenas 5 opções para a componente simétrica).

13.6 Handshake Protocol

Responsabilidades: negociação de parâmetros, autenticação, estabelecimento de chaves.

Exemplo: Handshake com RSA (TLS 1.2)

1. **ClientHello**: cliente → servidor (versão TLS, lista de cipher suites, `client_random`).
2. **ServerHello**: servidor → cliente (cipher suite escolhida, `server_random`).
3. **Certificate**: servidor → cliente (certificado digital do servidor).
4. **ServerHelloDone**: servidor → cliente (indica fim da fase "hello").
5. **ClientKeyExchange**: cliente → servidor (`pre_master_secret` cifrado com a chave pública do servidor).
6. Ambos derivam o `master_secret` = função_hash(`pre_master_secret`, `client_random`, `server_random`).
7. A partir do `master_secret`: derivam IVs, chaves de cifra e MAC para cada direção.
8. **ChangeCipherSpec**: cliente → servidor (a partir daqui usa os esquemas negociados).
9. **Finished**: cliente → servidor (HMAC de todas as mensagens do handshake, usando as novas chaves).
10. Servidor verifica e envia as suas próprias mensagens **ChangeCipherSpec** e **Finished**.

Autenticação do Servidor

- O cliente cifra o `pre_master_secret` com a chave pública do servidor (do certificado).
- Apenas o servidor legítimo (com a chave privada correta) consegue decifrar.
- A mensagem `Finished` com HMAC válido prova que o servidor conhece o `pre_master_secret` correto.

Autenticação do Cliente (opcional)

- Servidor envia `CertificateRequest` após o seu `Certificate`.
- Cliente envia o seu `Certificate` e uma mensagem `CertificateVerify` (assinatura digital de todas as mensagens de handshake com a chave privada do cliente).

Mudanças no TLS 1.3

- Eliminação do RSA puro para estabelecimento de chaves (não fornece *Perfect Forward Secrecy*).
- Informações do `ClientKeyExchange` incorporadas diretamente no `ClientHello` (reduz latência).

13.7 HTTPS

- HTTP sobre TLS; porta padrão: 443.
- Verificação adicional: o hostname da URI deve corresponder ao `Common Name` ou ao `Subject Alt Names` (DNS Name) do certificado do servidor.
- Suporta *wildcards* no `Common Name` (ex: `*.sapo.pt`).

13.8 Derivação de Chaves

Simplificadamente:

1. `pre_master_secret` + `client_random` + `server_random` → (função hash) → `master_secret`.
2. `master_secret` → (outra função hash) → IVs, chaves de cifra e MAC para cada direção.

13.9 Perfect Forward Secrecy (PFS)

Sem PFS: se o atacante gravar o tráfego e depois obtiver a chave privada do servidor, pode decifrar tudo o que foi gravado (ex: Wireshark com chave privada).

Com PFS: mesmo obtendo a chave privada posteriormente, o atacante **não consegue decifrar** comunicações passadas, porque as chaves de sessão nunca foram transmitidas.

Razão pela qual RSA puro não tem PFS: o `pre_master_secret` é transmitido cifrado — com a chave privada, um atacante pode decifrar.

13.10 Método de Diffie-Hellman

Permite estabelecer um segredo compartilhado **sem transmiti-lo** pela rede.

Parâmetros públicos: p (primo grande) e g (raiz primitiva módulo p).

Processo:

1. Cliente escolhe a (secreto) e calcula $X = g^a \bmod p \rightarrow$ envia X ao servidor.
2. Servidor escolhe b (secreto) e calcula $Y = g^b \bmod p \rightarrow$ envia Y ao cliente.
3. Cliente calcula: $Z = Y^a \bmod p = g^{ab} \bmod p$.
4. Servidor calcula: $Z = X^b \bmod p = g^{ab} \bmod p$.
5. Ambos chegam ao mesmo segredo Z sem o terem transmitido.

Segurança: calcular a a partir de X = computar o **logaritmo discreto** $\rightarrow$ computacionalmente infazível para valores grandes de p .

Exemplo numérico ($p=11$, $g=6$):

- Servidor: $a=4$, $Y = 6^4 \bmod 11 = 9$.
- Cliente: $b=7$, $X = 6^7 \bmod 11 = 8$.
- Cliente: $Z = 9^7 \bmod 11 = 4$.
- Servidor: $Z = 8^4 \bmod 11 = 4$. ✓

Variante: ECDH (*Elliptic Curve Diffie-Hellman*) — usa curvas elípticas.

No TLS, o DH/ECDH garante PFS e são os únicos métodos permitidos no TLS 1.3.

13.11 Sockets TLS em Java

Classes principais:

- `SSLSocket` / `SSLServerSocket`: equivalentes TLS do `Socket` / `ServerSocket`.
- `SSLConnectionFactory` / `SSLServerSocketFactory`: factories para criação.

Fluxo de uso:

```
SSLContext sc = SSLContext.getInstance("TLSv1.2");
sc.init(null, null, null); // null = valores por omissão
SSLConnectionFactory factory = sc.getSocketFactory();
SSLSocket client = (SSLSocket) factory.createSocket("host", 443);
```



```
client.startHandshake();
SSLSession session = client.getSession();
```

SSLContext: configuração da implementação TLS. Recebe:

- **KeyManager:** material criptográfico para autenticação (chaves privadas).
- **TrustManager:** políticas de confiança (quais trust anchors usar).
- **SecureRandom:** fonte de aleatoriedade.

14. Autenticação Baseada em Passwords

14.1 Identificação vs. Autenticação

- **Identificação:** definição de uma informação que estabelece quem é um utilizador (ex: username, email).
- **Autenticação:** verificação/prova da identidade alegada.

14.2 Fatores de Autenticação

Fator	Descrição	Exemplos
Conhecimento	Algo que só o utilizador conhece	passwords, passphrases
Propriedade	Algo que só o utilizador possui	tokens criptográficos, acesso a telemóvel
Característica	Características biométricas	impressão digital, íris

Autenticação de múltiplos fatores: combina dois ou mais fatores, reduzindo a probabilidade de sucesso de um atacante.

14.3 Funcionamento de um Sistema de Autenticação

Não se armazena a password diretamente. Armazena-se a **informação de validação** $v = f(a)$, onde:

- a = informação de autenticação (a password).
- $f(.)$ = função que transforma a em v (tipicamente uma função hash criptográfico).

Validação: $g(v)(a) = (v = f(a))$ — compara a hash da password fornecida com a hash armazenada.

Exemplo Linux: `/etc/shadow` armazena hashes das passwords; `/etc/passwd` contém apenas informações básicas.

14.4 Tipos de Ataques de Dicionário

Tipo	Condição	Descrição
Tipo 2	Sem acesso a v	Atacante usa a interface de autenticação para testar passwords
Tipo 1	Com acesso a v	Mais eficiente: calcula $f(a)$ para cada entrada do dicionário e compara com v

Ataques com pré-computação (Rainbow Tables): pré-computar para cada password a_i do dicionário o par $(f(a_i), a_i)$ e armazenar num array. O ataque resume-se a uma simples consulta: $M[v]$.

14.5 Contramedidas

Para ataques de Tipo 2 (interface)

- **Passwords complexas**: longas, misturando letras, números e caracteres especiais.
- **Backoff**: atraso crescente após falhas consecutivas.
- **Terminação da ligação**: terminar a ligação após N falhas.
- **Bloqueamento**: bloquear o acesso temporária ou permanentemente.
- **Jailing**: permitir acesso mas com funcionalidade limitada.
- **Testes de Turing / CAPTCHA**: impedir automatização do ataque.
- **Desafios computacionais**: aumentar o custo por tentativa.

Para ataques de Tipo 1

- **Limitar acesso à base de validação**.
- **Aumentar o custo de $f(\cdot)$** : funções hash propositalmente lentas.
- **Salt**: adiciona um valor aleatório ao cálculo do hash.

14.6 Salt

Quando o utilizador u é registado, sorteia-se aleatoriamente um salt_u específico. A informação de validação é: $v_u = f(\text{salt}_u \parallel a_u)$. Ambos salt_u e v_u são armazenados.

Proteção: como cada utilizador tem um salt diferente, um atacante não pode usar uma única *Rainbow Table* para atacar todos os utilizadores — teria que criar tabelas específicas para cada salt possível, o que é inviável.

Formato no `/etc/shadow` (exemplo): $\text{\$y\$j9T\$<salt>\$<hash>}$

- y = função hash *yescrypt*.
- $j9T$ = parâmetros adicionais.
- $\text{KFXzGy0RXYho3ReDdB0P7}$ = salt.
- Valor final = hash.

14.7 Bcrypt

Sistema especializado para armazenamento de passwords:

- Computacionalmente **mais intensivo** que SHA-256 (propositalmente lento).
- Parâmetro de **custo** configurável (número de iterações).
- Gera automaticamente um salt aleatório.
- Formato: $\text{\$2b\$<custo>\$<22 chars de salt><31 chars de hash>}$.

```
bcrypt.hash(password, saltRounds, (err, hash) => { /* ... */ });
bcrypt.compare(password, storedHash, (err, result) => { /* ... */ });
```

14.8 Recomendações Atuais (OWASP)

Função hash recomendada: **Argon2id** (vencedora do *Password Hashing Competition*; especificada na RFC 9106).

Características do Argon2id:

- Alta complexidade de **espaço** (memória): resiste a ataques com GPU.
- Parâmetros: m (tamanho de memória), t (iterações), p (paralelismo).

Parametrizações mínimas recomendadas (todas com $p=1$):

- $m=47104$ (46 MiB), $t=1$
- $m=19456$ (19 MiB), $t=2$
- $m=12288$ (12 MiB), $t=3$
- $m=9216$ (9 MiB), $t=4$
- $m=7168$ (7 MiB), $t=5$

15. Protocolo HTTP e Cookies

15.1 Revisão do HTTP

O HTTP é o protocolo da aplicação web. Características:

- **Mensagens texto** (ASCII; a partir de HTTP/2, headers comprimidos em binário).
- **Cliente-servidor**: cliente = browser (requisições), servidor = responde.
- Porta padrão: **80** (HTTP) / **443** (HTTPS).
- Protocolo de transporte: **TCP** (até HTTP/2); **QUIC/UDP** (HTTP/3).
- **Stateless**: cada requisição é tratada independentemente.

15.2 Formato de Mensagens HTTP

Requisição

```
GET /caminho HTTP/1.1
Host: www.exemplo.pt
User-Agent: curl/7.16.3
Accept-Language: pt, en
[linha em branco]
[corpo - opcional]
```

- **Request line**: método + caminho + versão HTTP.
- **Request headers**: metadados (pares nome: valor).
- **Linha em branco**: separa headers do corpo.
- **Corpo**: payload (vazio em GETs, presente em POSTs).

Métodos principais:

- **GET**: obter recurso.
- **HEAD**: apenas headers do recurso.

- **POST**: enviar dados para o servidor.

Resposta

```
HTTP/1.1 200 OK
Content-Type: text/html
[linha em branco]
[corpo]
```

Códigos de status:

- **1XX**: informação.
- **2XX**: sucesso (ex: **200 OK**).
- **3XX**: redireção (ex: **301 Moved Permanently**, **302 Found**).
- **4XX**: erro do cliente (ex: **403 Forbidden**, **404 Not Found**).
- **5XX**: erro do servidor (ex: **500 Internal Server Error**).

15.3 Redireções (302)

```
HTTP/1.1 302 Found
Location: /novo_caminho
```

O browser automaticamente faz uma nova requisição para a URI especificada no **Location**. Muito usado em mecanismos de autenticação.

15.4 Cookies

Definição: pedaço de informação (par **nome=valor**) que o servidor pede ao cliente para armazenar e incluir em requisições futuras.

Criação (via header **Set-Cookie**):

```
Set-Cookie: session_id=12345678; Expires=...; Secure; HttpOnly; SameSite=Strict
```

Envio pelo cliente (via header **Cookie**):

```
Cookie: session_id=12345678; app_version=2.1
```

Propriedades de Segurança dos Cookies

Propriedade	Efeito
Expires / Max-Age	Data/duração de validade

Propriedade	Efeito
Domain	Domínios/subdomínios onde o cookie é enviado
Path	Restringe o cookie a certos caminhos
Secure	Cookie só é enviado sobre HTTPS
HttpOnly	Cookie não é acessível via JavaScript (<code>document.cookie</code>)
SameSite=Strict	Cookie não é enviado em requisições <i>cross-site</i>

Third-Party Cookies

- Cookies criados por domínios diferentes do site que se está a visitar.
- Uso legítimo: autenticação por terceiros.
- Uso problemático: **tracking** de navegação para construção de perfis.
- Regulados pela GDPR na UE: sites devem obter consentimento do utilizador.

Sequestro de Sessão

- Se um atacante obtém o valor de um cookie de sessão de um utilizador autenticado, pode configurar o seu browser com esse valor e aceder como se fosse aquele utilizador.
- **Demonstração:** editar o valor do cookie `MoodleSession` no browser.

16. Autenticadores em Contexto Web

16.1 Autenticação Basic do HTTP

```
Authorization: Basic YWxpY2U6MTIzNDU2
```

O valor é `username:password` codificado em Base64.

Problema (stateless): o browser deve incluir as credenciais em **todas** as requisições a recursos protegidos, obrigando a armazenar a password.

Quando o servidor recebe uma requisição sem credenciais para um recurso protegido, responde com:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Basic realm=www.exemplo.pt
```

16.2 Conceito de Autenticador

Alternativa ao *Basic Authentication*:

Fase 1: utilizador envia credenciais → servidor valida → gera **autenticador** → envia ao cliente.

Fase 2: cliente inclui o autenticador em cada requisição → servidor verifica validade → autoriza ou não.

Requisito: o autenticador deve ser **verificável** (não forjável por atacantes).

16.3 Ataques Relacionados a Autenticadores

- **Falsificação existencial:** obter qualquer autenticador válido.
- **Falsificação seletiva:** obter autenticador para um utilizador específico.
- **Obtenção da chave do servidor:** possibilita forjar autenticadores para qualquer utilizador.

16.4 Autenticadores com Cookies

Abordagem 1: Identificador de Sessão

- Após autenticação bem-sucedida, gera-se um **identificador aleatório** (ex: 32 bytes aleatórios).
- As informações da sessão (incluindo identidade do utilizador) são armazenadas **no servidor** indexadas pelo identificador.
- O cookie contém apenas o identificador.
- **Verificação:** o servidor verifica se o identificador existe na sua base de sessões.
- **Requisito de segurança:** o identificador deve ser **imprevisível** (gerado aleatoriamente com grande dimensão).

Abordagem 2: MAC

- O autenticador contém as informações de sessão cifradas ou protegidas com MAC.
- O servidor usa uma chave simétrica secreta para gerar/verificar a marca.
- As informações de sessão estão **no próprio cookie** (servidor não precisa de armazenar estado).
- **Verificação:** recalculando o MAC e comparar.

16.5 Aspectos Adicionais dos Autenticadores

Validade Temporal

- A data de expiração do cookie **não basta** (atacantes podem ignorá-la).
- Deve-se incluir informações de validade no próprio autenticador (protegidas por MAC ou no lado servidor).

Revogação

- **Com identificador de sessão:** basta invalidar/remover a sessão na base de dados do servidor.
- **Com informações no cookie:** necessário manter uma **lista de revogação** (o servidor verifica se o autenticador consta desta lista).

Configuração de Segurança dos Cookies de Autenticação

Devem sempre ter: **Secure**, **HttpOnly**, **SameSite=Strict**.

16.6 Cross-Site Request Forgery (CSRF)

Ataque:

1. Utilizador está autenticado na Aplicação B (tem cookie válido).

17.1 Motivação

Problema: como autorizar uma aplicação a aceder a recursos de um utilizador num servidor remoto, sem fornecer as credenciais do utilizador à aplicação?

Objetivos:

- Restringir os recursos acessíveis pela aplicação.
- Evitar que o utilizador forneça as suas credenciais à aplicação.
- Permitir revogar a autorização sem afetar outras autorizações.

17.2 Papéis (RFC 6749)

Papel	Descrição	Exemplo
Resource Owner	Proprietário dos recursos; autoriza o acesso	Alice
Resource Server	Servidor com os recursos protegidos	myphotos.com
Client	Aplicação que requisita acesso	printonline.com
Authorization Server	Autentica o Resource Owner; emite Access Tokens	Servidor de autorização do myphotos.com

17.3 Tipos de Client

- **Confidencial:** capaz de armazenar credenciais em segredo (ex: aplicação web clássica no servidor). Recebe `client_id` + `client_secret`.
- **Público:** incapaz de proteger credenciais (ex: app no browser, app nativa). Recebe apenas `client_id`.

17.4 Registo de Clients

Antes de usar o OAuth 2.0, a aplicação **client** deve ser registada junto ao Authorization Server.

Informações obrigatórias (pelo OAuth 2.0):

- Tipo de client (confidencial/público).
- **URL de redirecionamento** (`redirect_uri`).

Resultado do registo:

- `client_id` (sempre).
- `client_secret` (apenas para clients confidenciais).

17.5 Access Token

- Credencial de acesso ao Resource Server.
- Formato opaco para o client.
- Contém informação sobre os *scopes* autorizados.
- Verificável pelo Resource Server (autenticidade/integridade garantida).
- Frequentemente **Bearer Tokens** (incluídos via header `Authorization: Bearer <token>`).

17.6 Scopes

Strings que descrevem o tipo de autorização solicitada pelo client.

- Não definidos pelo OAuth 2.0 — cada implementação define os seus.
- Exemplo Google: "<https://www.googleapis.com/auth/tasks>".
- Múltiplos scopes separados por espaço.
- O Authorization Server pode autorizar apenas parcialmente.

17.7 Refresh Tokens

- Permitem ao client obter novos Access Tokens sem repetir o fluxo de autorização completo.
- Apresentados ao token endpoint com `grant_type=refresh_token`.
- O Authorization Server pode emitir novo refresh token (invalidando o anterior) ou manter o mesmo.

17.8 Fluxos de Autorização (*Authorization Grant Flows*)

1. Resource Owner Password Grant

- O client usa diretamente as credenciais do Resource Owner.
- **Não recomendado** (exceto para sistemas legados ou elevada confiança client-utilizador).
- Vantagem sobre uso direto das credenciais: o client deve descartá-las após obter o token; o token pode ter scopes limitados; acesso auditável e revogável.

2. Client Credentials Grant

- Apenas client e Authorization Server (sem Resource Owner ativo).
- O client autentica-se com as suas próprias credenciais.
- Apenas para clients confidenciais.
- Não permite refresh tokens.
- Pré-requisito: Resource Owner configura previamente as permissões.

```
POST /token HTTP/1.1
Authorization: Basic <client_credentials>
Content-Type: application/x-www-form-urlencoded
grant_type=client_credentials
```

3. Authorization Code Grant (**recomendado**)

Fluxo baseado em redirecionamento:

```
Client → (1) Redireciona user-agent para authorization endpoint
User-agent → (2) Authorization Server (autenticação + consentimento)
Authorization Server → (3) Redireciona user-agent para redirect_uri +
authorization code
Client → (4) Troca authorization code por access token (via back channel)
Authorization Server → (5) Access token (+ optional refresh token)
```

Parâmetros do passo (1) (`authorization endpoint`):

- `response_type=code` (obrigatório).
- `client_id` (obrigatório).
- `redirect_uri`.
- `scope`.
- `state` (recomendado — proteção CSRF).

Passo (4) (`token endpoint`):

```
POST /token HTTP/1.1
Authorization: Basic <client_credentials>
grant_type=authorization_code&code=<auth_code>&redirect_uri=<redirect_uri>
```

Vantagens do Authorization Code:

- Client autentica-se antes de receber o access token.
- Access token não passa pelo user-agent (não exposto ao Resource Owner).
- Proteção CSRF via parâmetro `state`.

4. Implicit Grant

- Simplificação do Authorization Code: o access token é entregue diretamente na redirect URI (sem troca de authorization code).
- Menos seguro; não recomendado em geral.
- Mais rápido (menos passos).

17.9 Front Channel vs. Back Channel

- **Front Channel:** comunicação via redirecionamentos (através do user-agent). Usado no `authorization endpoint`. Informações visíveis ao Resource Owner; não adequado para credenciais do client.
- **Back Channel:** comunicação direta client ↔ Authorization Server. Usado no `token endpoint`. Credenciais do client transmitidas com segurança.

18. OpenID Connect

18.1 Conceito

Extensão do OAuth 2.0 que adiciona uma camada de **autenticação**. Permite que uma aplicação cliente (RP) autentique um utilizador (end-user) junto a um fornecedor de identidade (OP).

Abstratamente: é um *scope* adicional do OAuth 2.0 (**openid**) que autoriza o acesso a informações de identidade do utilizador.

18.2 Papéis

Papel	Nome Alternativo	Descrição
End-User	<i>End-User</i>	Utilizador humano a autenticar
Aplicação Cliente	RP (<i>Relying Party</i>)	A "parte que confia" nas informações de identidade
Fornecedor de Identidade	OP (<i>OpenID Provider</i>)	Armazena registos de utilizadores; emite tokens de identidade

O OP combina as funções de Authorization Server + Resource Server do OAuth 2.0.

18.3 ID Token

- Token especial (JWT assinado pelo OP) que confirma a identidade do end-user.
- **Diferença do Access Token:** o ID Token tem formato standard interpretável pela RP; o Access Token é opaco para o client.
- Entregue juntamente com o Access Token.

Claims obrigatórias do ID Token:

- **iss:** URL do emitente.
- **sub:** identificador único do utilizador naquele OP (string, case-sensitive, <256 chars ASCII).
- **aud:** audiência (inclui o **client_id** da RP).
- **exp:** data de expiração (segundos desde Unix epoch).
- **iat:** data de emissão.
- **nonce** (obrigatório se enviado na requisição): previne ataques de repetição.

18.4 Fluxo Authorization Code

Resumo:

1. End-user acede a recurso protegido na RP.
2. RP redireciona user-agent para o **authorization endpoint** do OP com **scope=openid** (e opcionalmente outros scopes).
3. OP autentica o end-user (ex: formulário username/password).
4. OP solicita consentimento do end-user para partilhar informações com a RP.
5. OP redireciona user-agent de volta para **redirect_uri** da RP com authorization code.
6. RP troca authorization code por **ID Token + Access Token** (via back channel).
7. RP verifica a assinatura do ID Token e as claims.
8. RP cria sessão para o end-user (autenticado).
9. RP pode usar o Access Token para aceder ao **UserInfo endpoint**.

Requisição ao token endpoint (igual ao OAuth 2.0):

```
POST /token HTTP/1.1
Authorization: Basic <client_credentials>
grant_type=authorization_code&code=<auth_code>&redirect_uri=<redirect_uri>
```

Resposta inclui `id_token` adicional:

```
{
  "access_token": "SlAV32hkKG",
  "token_type": "Bearer",
  "id_token": "eyJhbGciOi..."
}
```

18.5 UserInfo Endpoint

- Recurso do OP que fornece informações adicionais sobre o end-user.
- Acedido pelo client com o Access Token:

```
GET /userinfo HTTP/1.1
Authorization: Bearer <access_token>
```

- Resposta: JSON com claims (ex: `name`, `email`, `picture`, `gender`, ...).
- Usa sempre Bearer tokens.

18.6 Combinação de Autenticação e Autorização

O OpenID Connect suporta combinar autenticação (scope `openid`) com autorização OAuth 2.0 num único fluxo, incluindo múltiplos scopes na requisição.

19. Ataques ao OAuth 2.0 e Contramedidas

19.1 Modelo de Atacante (RFC 6819)

1. Acesso total à rede entre client e authorization/resource server (mas não entre authorization e resource servers).
2. Recursos ilimitados.
3. Duas das três partes (client, resource owner, authorization server) podem estar em conluio.

19.2 Ameaças Contra o Client

Comprometimento do `client_secret`:

- Causas: inclusão no código-fonte, ficheiros de configuração acessíveis, etc.
- Consequências: impersonação do client legítimo, uso de refresh tokens capturados.
- **Demonstração:** o comando `strings` em binários revela constantes em texto plano.

Contramedidas:

- Credenciais **específicas por ambiente** de implantação (nunca no código-fonte).
- Armazenar em ficheiros de configuração com permissões restritas.
- Ter mecanismos de **revogação** do `client_secret`.

Comprometimento de Access Tokens:

- São frequentemente Bearer Tokens → quem os possui pode usá-los.
- **Contramedidas:** TLS nas comunicações, mínimo scope necessário, curta validade, permissões de ficheiro adequadas.

Client malicioso (phishing):

- Cria réplica da interface de autenticação do Authorization Server.
- OAuth 2.0 não tem proteção nativa contra isto.

19.3 Ameaças Contra o Authorization Endpoint

- **Phishing do Authorization Server:** atacante cria servidor falso. Contramedida: TLS/HTTPS e certificados digitais.
- **Scope excessivo:** client malicioso solicita mais permissões do que necessário. Contramedida: apresentação clara das permissões; pré-classificação de clients.

19.4 Ameaças Contra o Token Endpoint

- **Acesso à base de dados:** obtenção de credenciais de clients ou de tokens. Contramedida: políticas de segurança adequadas; não armazenar credenciais em claro (usar hashes).
- **Monitorização:** captura de credenciais/tokens em trânsito. Contramedida: TLS.
- **Força bruta sobre `client_secret`:** Contramedidas: secrets longos e aleatórios; limitação de tentativas; métodos de autenticação mais fortes (ex: certificados).

19.5 Ataque CSRF ao OAuth 2.0**Mecanismo:**

1. Atacante inicia fluxo Authorization Code e obtém authorization code para **os seus próprios recursos**.
2. Interrompe o fluxo antes de entregar o code ao client.
3. Força o browser da vítima a aceder à URL de callback do client com o authorization code do atacante.
4. O client associa a sessão da vítima ao authorization code do atacante → vítima acede aos recursos do atacante.

Consequência: dados da vítima são armazenados na conta do atacante (e.g., fotos editadas vão para conta do atacante).

Proteção: parâmetro `state`

1. Quando o client redireciona para o authorization endpoint, inclui um `state` (ex: hash do identificador de sessão do user-agent).
2. O authorization server inclui o mesmo `state` na URL de callback.
3. O client verifica se o `state` recebido corresponde ao `state` associado à sessão atual do user-agent.

4. Se não corresponder → CSRF detetado → rejeição.

Requisito: o valor do **state** deve ser **imprevisível** e **não forjável** pelo atacante.

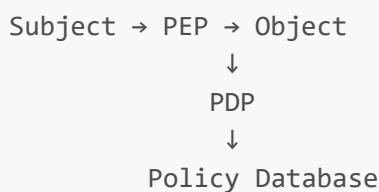
20. Controlo de Acesso e Modelos

20.1 Conceito de Controlo de Acesso

Controlo de Acesso: políticas e mecanismos empregados por um sistema para **restringir e auditar o acesso de entidades a recursos**.

Inclui: identificação, autenticação, autorização e **auditoria** (geração de logs).

20.2 Arquitetura: Monitor de Referências



- **PEP** (*Policy Enforcement Point*): intercepta operações e solicita decisão ao PDP. Deve ser inalterável, não contornável e avaliável (simples).
- **PDP** (*Policy Decision Point*): consulta as políticas e decide se autoriza ou não.

Relação com outros conceitos: PEP relaciona-se com os mecanismos de autenticação/autorização estudados; PDP relaciona-se com os modelos de controlo de acesso.

20.3 Modelo de Matriz de Acessos

Representação tabular: **linhas = sujeitos, colunas = objetos, células = permissões**.

	File 1	File 2	File 3	Program 1
Alice	read/write	read/write	—	execute
Bob	read	—	read/write	—
Charlie	—	read	—	execute/write

Não prevê permissões negativas.

Implementação por Tabela de Autorização

Lista de triplos (**Sujeito**, **Permissão**, **Objeto**). Simples, mas ineficiente para buscas por sujeito ou objeto.

Implementação por *Capabilities*

Cada sujeito tem uma lista das suas permissões sobre cada objeto. Eficiente para verificar permissões de um sujeito específico; ineficiente para "quem tem acesso a este objeto?".

Implementação por ACL (*Access Control List*)

Cada objeto tem uma lista de sujeitos e as suas permissões. Eficiente para obter permissões de um objeto; ineficiente para buscas por sujeito.

20.4 Permissões por Grupos

Extensão do modelo: permissões atribuídas a **grupos** de utilizadores. Utilizadores herdam as permissões do(s) grupo(s) a que pertencem. Permite representações mais compactas e gestão mais simples.

Exemplo (Linux/UNIX): cada ficheiro tem owner, grupo associado e bitmaps de permissões para owner, grupo e "outros" (3 bits: leitura, escrita, execução).

20.5 RBAC (Role-Based Access Control)

Motivação

- Permissões associadas a **papéis** (*roles*), não diretamente a utilizadores.
- Captura relações organizacionais (cargos, responsabilidades).
- Papéis e permissões mudam pouco; utilizadores podem transitar entre papéis.

Grupos vs. Papéis

- **Grupo:** coleção de utilizadores; pertença é intrínseca e fixa.
- **Papel:** conjunto de permissões; utilizadores são atribuídos a papéis que podem ativar/desativar por sessão.

Princípio do Privilégio Mínimo

Utilizadores devem ativar, numa sessão, apenas os papéis necessários para as tarefas a realizar.

20.6 RBAC₀ (Modelo Base)

Conjuntos:

- U : utilizadores.
- R : papéis.
- P : permissões.
- S : sessões ativas.

Relações:

- $UA \subseteq U \times R$: afetação de utilizadores a papéis (muitos-para-muitos).
- $PA \subseteq P \times R$: afetação de permissões a papéis (muitos-para-muitos).
- $users(s_i)$: função que associa cada sessão ao respetivo utilizador.
- $roles(s_i) \subseteq \{r \mid (users(s_i), r) \in UA\}$: subconjunto de papéis ativados pelo utilizador na sessão.

Nota: RBAC **não prevê permissões negativas**.

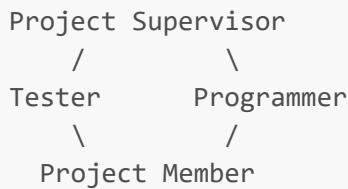
20.7 RBAC₁ (Hierarquia de Papéis)

Adiciona a relação *RH* (*Role Hierarchy*): pares de papéis sénior-júnior.

Notação: $Rx \succcurlyeq Ry$ significa "Rx é sénior de Ry" (Rx herda as permissões de Ry).

Efeito: papéis sénior herdam automaticamente todas as permissões dos papéis júnior.

Exemplo:



$roles(s_i)$ no RBAC₁: $\{r \mid \exists r' \succcurlyeq r, (users(s_i), r') \in UA\}$.

Private Roles

Para evitar que um papel sénior herde certas permissões específicas de papéis júnior:

- Cria-se um *private role* (ex: **Tester'**) que é sénior do papel original (**Tester**).
- As permissões que não devem ser herdadas são migradas para o *private role*.
- O papel sénior (**Project Supervisor**) herda apenas as permissões de **Tester**, não de **Tester'**.

20.8 RBAC₂ (Restrições)

Construído sobre RBAC₀ (sem hierarquia). Adiciona a possibilidade de **restrições**.

Tipos de restrições:

Tipo	Aplicação	Descrição
Cardinalidade	Relação <i>UA</i>	Limita o número de utilizadores num papel (ex: só 1 presidente)
Pré-requisitos	Relação <i>UA</i>	Um papel só pode ser atribuído se outro já o for (emula hierarquia)
Exclusão mútua estática	Relação <i>UA</i>	Um utilizador não pode ter dois papéis conflitantes simultaneamente
Exclusão mútua dinâmica	Função <i>roles(.)</i>	Um utilizador não pode <i>ativar</i> dois papéis conflitantes na mesma sessão

Separação de deveres: a exclusão mútua implementa este princípio organizacional (ex: quem solicita compras $\neq$ quem autoriza pagamentos).

20.9 RBAC₃

Combina RBAC₁ + RBAC₂: **hierarquia + restrições** (incluindo restrições sobre a hierarquia).

20.10 Controlo de Acesso no Windows

- **Access Token**: atribuído no login; contém SID do utilizador e lista de grupos.
- **Security Descriptor** de cada objeto:
 - **DACL** (*Discretionary ACL*): lista de ACEs (**Access Control Entry**). Cada ACE: (**SID**, **ações**, **resultado**: **allowed/denied**). Processadas **em ordem**; para na primeira correspondência.
 - **SACL** (*System ACL*): para auditoria (registo de tentativas de acesso em log).

Regras DACL:

- Sem DACL: acesso irrestrito.
- DACL vazia: negação irrestrita.

20.11 Controlo de Acesso e Legislação

A **Resolução do Conselho de Ministros n.º 41/2018** (relacionada com o RGPD) impõe:

- Criação de perfis de utilizador com **privilégios mínimos**.
- **Registo de tentativas** de acesso não autorizadas.
- Geração de **alarmes** após múltiplas tentativas falhadas.
- **Registo de todas as ações** de cada utilizador sobre dados pessoais.

20.12 ABAC (Attribute-Based Access Control)

Alternativa ao RBAC: permissões baseadas em **atributos** de utilizadores, objetos e condições ambientais.

Atributos de utilizadores: nome, idade, cargo, ter ou não certa aplicação instalada.

Atributos de objetos: nome, tipo, owner, data de modificação, classificação.

Condições ambientais: hora do dia, localização do acesso, tipo de dispositivo.

Vantagens sobre RBAC:

- Evita "explosão de papéis" para representar condições complexas.
- Mais flexível e expressivo.

Exemplo: um terminal multibanco com limite variável conforme o horário não precisa de papéis separados — o horário é apenas uma condição ambiental na regra.

É possível **combinar** RBAC e ABAC: usar a estrutura de papéis do RBAC complementada com atributos onde estes simplificam as políticas.

21. Biblioteca Casbin

21.1 Visão Geral

- Suporta múltiplos modelos: RBAC, ABAC, ACL, etc.
- Implementa o **PDP** (não o PEP — este é da responsabilidade da aplicação).
- Políticas definidas em **ficheiros externos** (não no código-fonte).

21.2 Ficheiros de Configuração

Ficheiro	Conteúdo
<code>model.conf</code>	Define o modelo (formato das requisições, permissões, matchers)
<code>policy.conf</code>	Define as políticas concretas (permissões, atribuições de papéis, hierarquia)

21.3 O Objeto `Enforcer`

```
const { newEnforcer } = require('casbin');

const pdp = async function(s, o, a) {
  const enforcer = await newEnforcer('model.conf', 'policy.conf');
  const result = await enforcer.enforce(s, o, a);
  return result; // true = permitido, false = negado
}
```

Parâmetros do `enforce()`: sujeito (`s`), objeto (`o`), ação (`a`).

Como middleware:

```
const rbac = (req, resp, next) => {
  const sub = req.session.user;
  const { path: obj } = req;
  const act = req.method;
  newEnforcer('model.conf', 'policy.conf').then(enforcer => {
    enforcer.enforce(sub, obj, act).then(decision => {
      if (decision) next();
      else resp.redirect('/unauthorized');
    });
  });
};

app.use(rbac);
```

21.4 Formato do `model.conf`

Exemplo para ACL

```
[request_definition]
r = sub, obj, act

[policy_definition]
p = sub, obj, act

[policy_effect]
e = some(where (p.eft == allow))
```

```
[matchers]
m = r.sub == p.sub && r.obj == p.obj && r.act == p.act
```

Adição para RBAC

```
[role_definition]
g = _, _

[matchers]
m = g(r.sub, p.sub) && r.obj == p.obj && r.act == p.act
```

A notação `g(r.sub, p.sub)` verifica se o sujeito é o próprio, tem o papel diretamente ou herda o papel (via hierarquia).

Secção [policy_effect]: o `p.eft` por omissão é `allow`. Suportados apenas alguns valores predefinidos.

Secção [matchers]: suporta operadores lógicos (`&&`, `||`, `!`) e aritméticos (`+`, `-`, `*`, `/`).

21.5 Formato do `policy.conf`

```
# Atribuição de papéis (g = role_definition)
g, alice, admin
g, admin, coordinator    # hierarquia: admin é sénior de coordinator

# Permissões (p = policy_definition)
p, coordinator, data1, write
p, alice, data1, read
p, bob, data2, write
```

- Linhas com `g`: atribuição utilizador→papel ou hierarquia papel_senior→papel_junior.
- Linhas com `p`: permissões (`sujeito, objeto, ação`).

21.6 Editor Online

Disponível em <https://casbin.org/editor/> — permite testar modelo e políticas interativamente com requisições específicas.

22. Termos-Chave

ABAC — *Attribute-Based Access Control*; modelo de controlo de acesso baseado em atributos.

ACL — *Access Control List*; lista de controlo de acesso associada a um objeto.

AES — *Advanced Encryption Standard*; primitiva de cifra de bloco (128 bits/bloco); standard atual do NIST.

ASN.1 — *Abstract Syntax Notation One*; linguagem para descrição de estruturas de dados independente de plataforma.

Autenticador — documento digital gerado pelo servidor após autenticação bem-sucedida, apresentado em requisições futuras.

Autoridade de Certificação (CA/AC) — entidade que verifica identidades e emite certificados digitais.

Bearer Token — token utilizável por qualquer entidade que o possui, sem necessidade de prova adicional.

Bcrypt — função de hash para armazenamento de passwords; computacionalmente intensa com custo configurável.

CBC — *Cipher Block Chaining*; modo de operação em cadeia para cifras de bloco.

Cipher Suite — conjunto de esquemas criptográficos usados numa ligação TLS.

Controlo de acesso — políticas e mecanismos para restringir e auditar o acesso a recursos.

CRIME — ataque que explora a compressão em TLS para inferir informações confidenciais.

CRL — *Certificate Revocation List*; lista de certificados revogados.

Criptograma — mensagem em texto cifrado.

CSRF — *Cross-Site Request Forgery*; ataque que força um browser a executar ações não intencionadas.

CTR — modo *Counter*; modo de operação em stream com contador como estado.

DES — *Data Encryption Standard*; primitiva de cifra de bloco obsoleta (56 bits de chave).

Diffie-Hellman — método de estabelecimento de chave partilhada sem transmitir o segredo pela rede; garante PFS.

DER — *Distinguished Encoding Rules*; regras de codificação binária para ASN.1.

DTLS — *Datagram TLS*; variante do TLS para UDP.

ECB — *Electronic Codebook*; modo de operação mais simples (sem encadeamento).

GDPR/RGPD — Regulamento Geral sobre a Proteção de Dados (UE).

GCM — *Galois Counter Mode*; modo que combina cifra e MAC.

Hash criptográfico — função que mapeia dados de tamanho arbitrário para um resumo de tamanho fixo, com propriedades de resistência a colisões e pré-imagens.

HMAC — *Hashed MAC*; esquema MAC baseado em funções hash.

HTTPS — HTTP sobre TLS; porta padrão 443.

ID Token — JWT assinado pelo OP (OpenID Provider) que confirma a identidade do utilizador.

IV (*Initialization Vector*) — valor aleatório usado em modos de operação para garantir que mensagens iguais geram criptogramas diferentes.

JCA — *Java Cryptography Architecture*; API criptográfica padronizada para Java.

JWT — *JSON Web Token*; formato standard para representar claims; composto por header.payload.signature.

Keystream — fluxo de bits pseudo-aleatórios gerados em modos de operação em stream.

Man-In-The-Middle (MITM) — ataque em que o atacante se interpõe na comunicação entre duas partes.

Nonce — número arbitrário usado uma única vez; previne ataques de repetição.

OAuth 2.0 — framework de autorização que permite a uma aplicação aceder a recursos protegidos de um utilizador.

OID — *Object Identifier*; esquema hierárquico de nomeação de objetos.

OpenID Connect — extensão do OAuth 2.0 que adiciona autenticação.

OP — *OpenID Provider*; fornecedor de identidade no OpenID Connect.

Padding — bits/bytes adicionados para ajustar o tamanho de uma mensagem ao bloco.

PDP — *Policy Decision Point*; componente que decide se uma operação é autorizada.

PEP — *Policy Enforcement Point*; componente que interceta operações e aplica decisões do PDP.

Perfect Forward Secrecy (PFS) — propriedade que impede a decifra de comunicações passadas mesmo com a chave privada comprometida posteriormente.

PEM — formato de armazenamento de material criptográfico em Base64.

PFX/PKCS#12 — container cifrado para chave privada + certificados.

PKI — *Public Key Infrastructure*; infraestrutura de chave pública.

PKCS — *Public Key Cryptography Standards*; conjunto de standards criptográficos da RSA Security.

RBAC — *Role-Based Access Control*; modelo de controlo de acesso baseado em papéis.

RP — *Relying Party*; aplicação cliente no OpenID Connect.

RSA — primitiva criptográfica assimétrica (Rivest, Shamir, Adleman); baseia-se na dificuldade de fatorar números grandes.

Salt — valor aleatório adicionado ao cálculo do hash de uma password para proteger contra ataques de pré-computação.

Scope — string que define o tipo de autorização solicitada no OAuth 2.0.

SHA-256 — função hash criptográfico (256 bits de output); amplamente recomendada.

Sequestro de sessão — ataque em que o atacante usa o cookie de sessão de um utilizador autenticado.

Texto plano — mensagem original não cifrada.

TLS — *Transport Layer Security*; protocolo criptográfico para comunicação segura.

Trust Anchor — certificado raiz implicitamente confiável; pré-instalado no sistema.

Vetor de inicialização — ver *IV*.

X.509 — standard da ITU que define a estrutura de certificados digitais.

Resumo produzido exclusivamente com base nos ficheiros Aula01.md a Aula20.md, sem recurso a fontes externas.